SURF detectors and descriptors

References

- SURF homepage http://www.vision.ee.ethz.ch/~surf/
- H. Bay, T. Tuytelaars, and L. V. Gool, SURF: Speeded Up Robust Features, ECCV 2006 http://www.vision.ee.ethz.ch/~surf/eccv06.pdf
- H. Bay, A. Ess, T. Tuytelaars, and L. V. Gool, SURF: Speeded Up Robust Features (SURF), Computer Vision and Image Understanding (CVIU), Vol. 110, No. 3, pp. 346–359, 2008 ftp://ftp.vision.ee.ethz.ch/publications/articles/eth_biwi_00517.pdf
- C. Evans, OpenSURF library http://www.chrisevansdev.com/computer-vision-opensurf.html

Interest operator

- **Interest point detector finds distinctive locations in image (corners, blobs, T-junctions)**
  - **should be *repeatable***
- **Interest point *descriptor* used for matching**
  - **should be *distinctive* and *robust***

Both should be *fast*

Harris features

- **Recall Harris features?**
  (Eigenvalues of gradient covariance matrix)

- **Dependent upon scale**

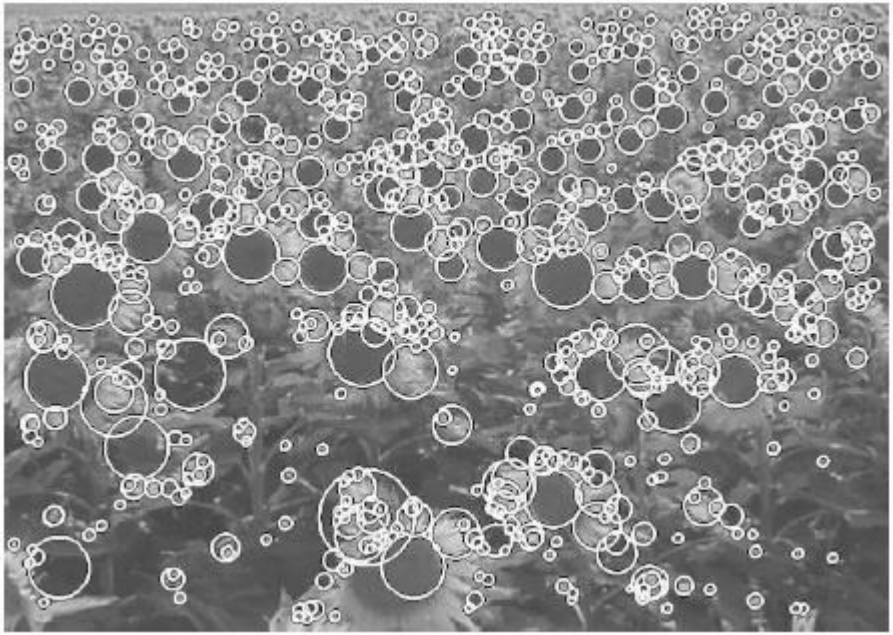
Scale space

- **Successively smooth an image...**

ftp://ftp.nada.kth.se/CVAP/reports/Lin08-EncCompSci.pdf

SIFT / SURF

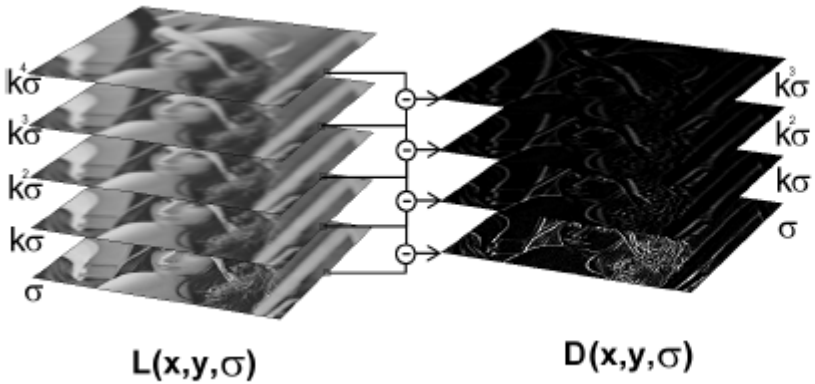
- SIFT – scale invariant feature transform (Lowe 2004)
- SURF – speeded up robust features (Bay et al. 2006)

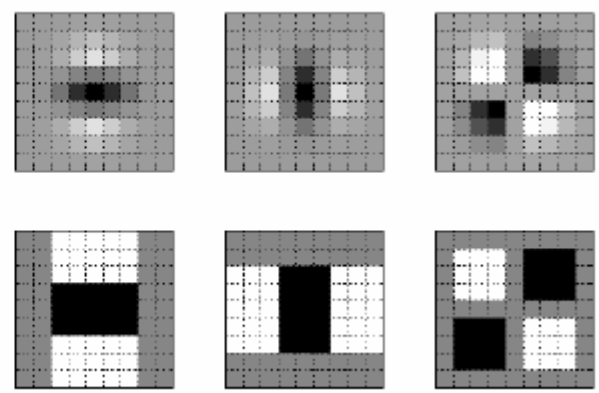


Introduction

- **SURF: is a local invariant interest point detector-descriptor for finding *correspondences* between two images of the same scene or object.**
- **SURF: is invariant to image rotation, scale changes, illumination change and small change in viewpoint.**

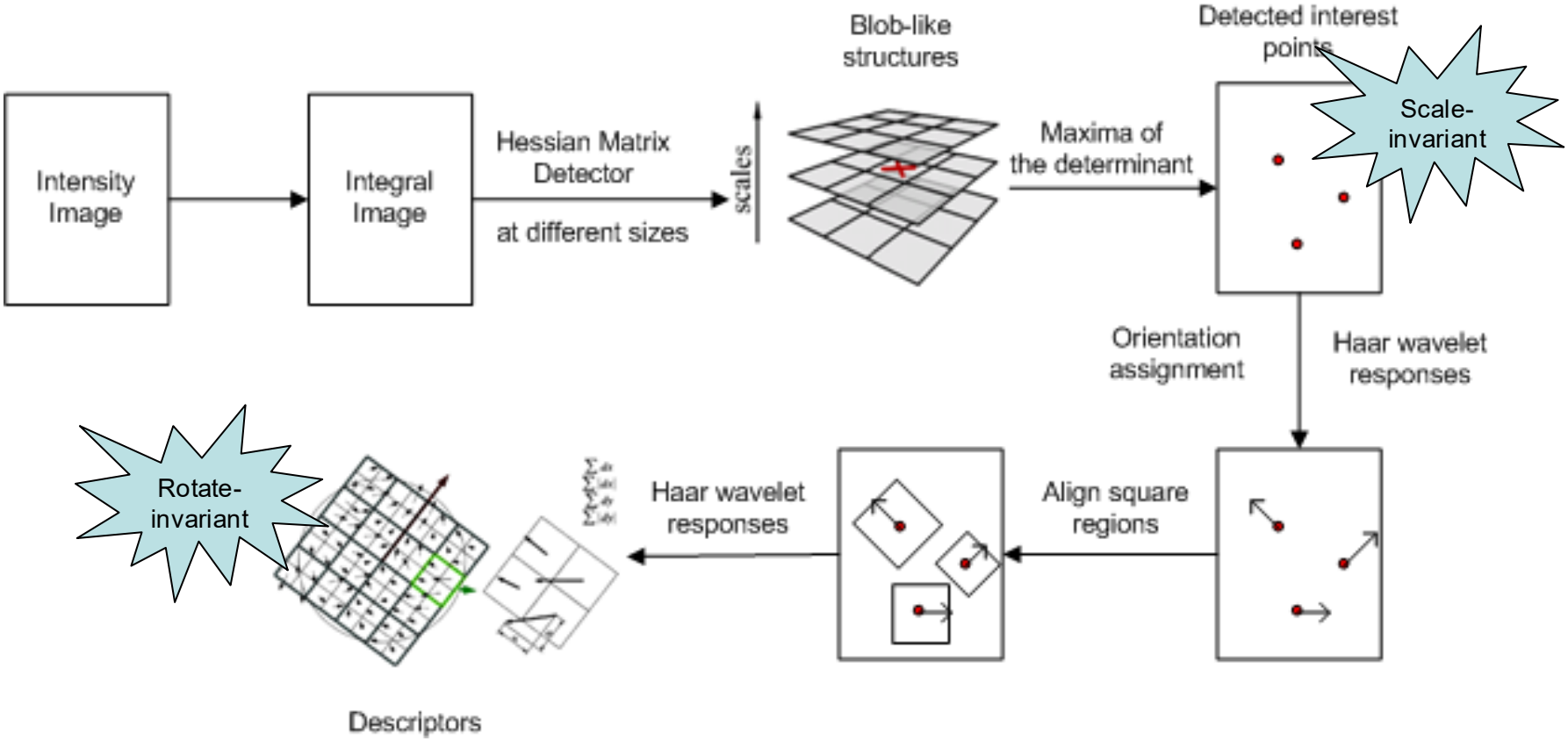
SIFT and SURF





	SIFT	SURF
Scale-space	Difference of Gaussians (approximates trace of Hessian)	Box filters (approximate determinant of Hessian)
Detector	Maxima & Minima (discard low-intensity extrema and points on edges)	Maxima (blob-detector), stores sign of the trace Hessian for matching step
Descriptor	Based on gradient magnitude	Based on Haar wavelet response

SURF flowchart



SURF algorithm

Interest point *detector*:

- **Compute integral image**
- **Apply 2nd derivative (approximate) filters to image**
- **Non-maximal suppression (Find local maxima in (x,y,σ) space)**
- **Quadratic interpolation**

SURF algorithm

Interest point *descriptor*:

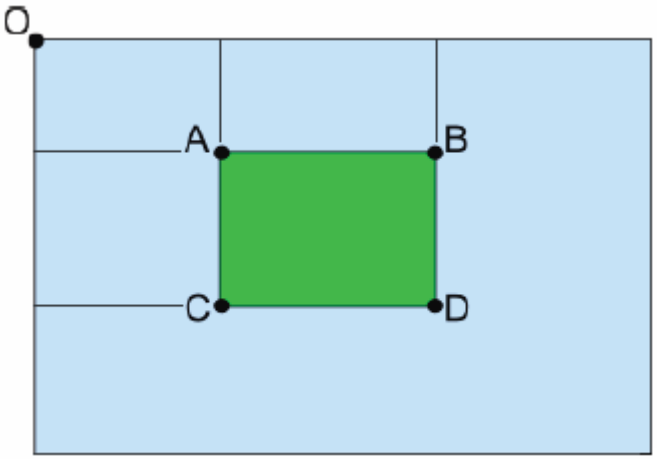
- **Divide window into 4x4 (16 subwindows)**
- **Compute Haar wavelet outputs**
- **Within each subwindow, compute**

$$v_{subregion} = \left[\sum dx, \sum dy, \sum |dx|, \sum |dy|\right]$$

- **This yields a 64-element descriptor**

(Only implement USURF – no rotation)

Integral image



- **Integral image (a.k.a. summed area table) is a 2D running sum**
- **$S(x,y) = \Sigma\Sigma\ I(x,y)$**
- **To compute,**

$$S(x,y) = I(x,y) - S(x-1,y-1) + S(x-1,y) + S(x,y-1)$$

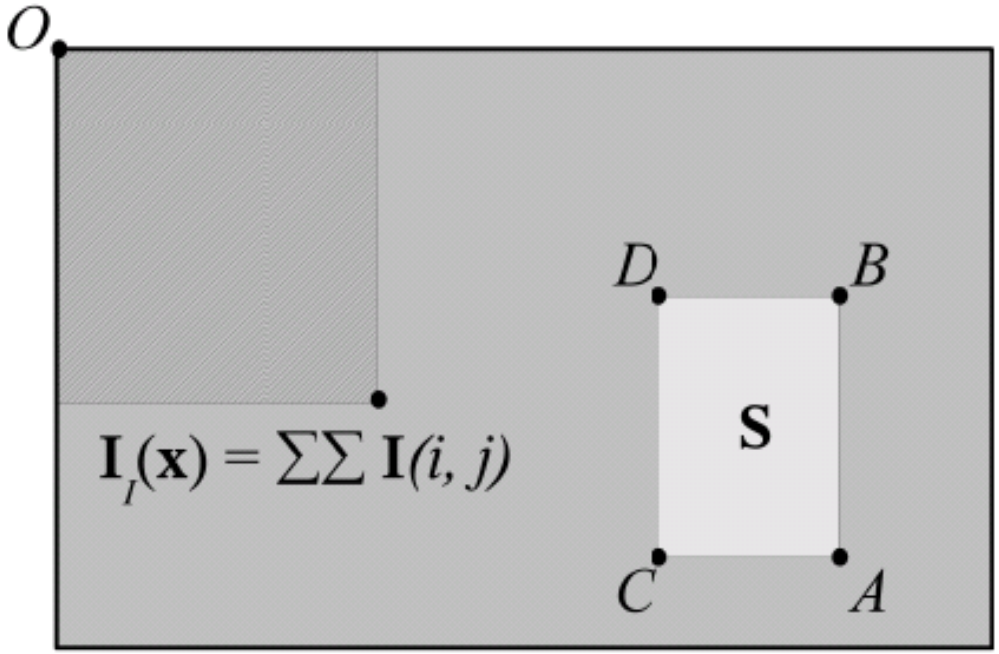
- **To use,**

$$V(l,t,r,b) = S(l,t) + S(r,b) - S(l,b) - S(r,t)$$

Returns sum of values inside rectangle

- **Note: Sum of values in *any* rectangle can be computed in constant time!**

Interest points Detection



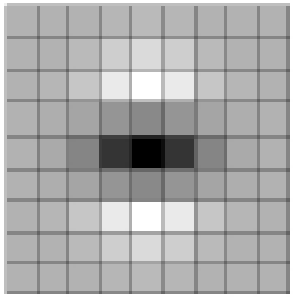
$$H = \begin{bmatrix} L_{xx} & L_{xy} \\ L_{xy} & L_{yy} \end{bmatrix}$$

$\mathbf{S} = A - B - C + D$

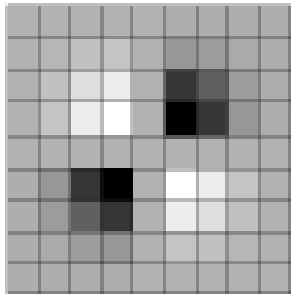
where $L_{xx}(\mathbf{x}, \sigma)$ is the convolution of the Gaussian second order derivative $\frac{\partial^2}{\partial x^2} g(\sigma)$ with the image I in point $\mathbf{x}$, and similarly for $L_{xy}(\mathbf{x}, \sigma)$ and $L_{yy}(\mathbf{x}, \sigma)$.

2nd derivative filters

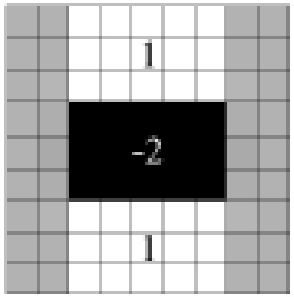
(9x9 filters)



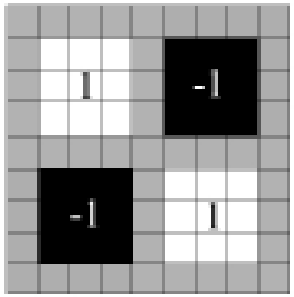
$$\frac{\partial^2 I}{\partial y^2}$$



$$\frac{\partial^2 I}{\partial x \partial y}$$



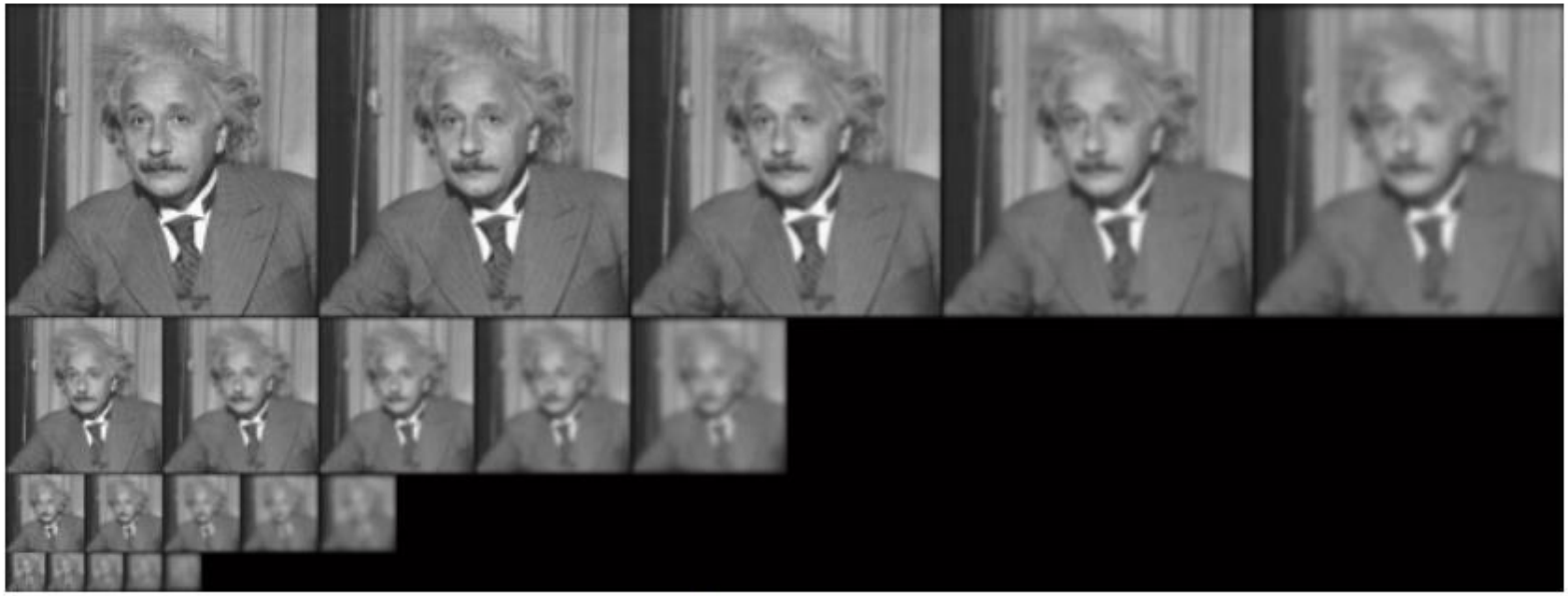
D_{yy}



D_{xy}

σ=1.2 scale s=1.2

$$\det(H_{approx}) = D_{xx}D_{yy} - (0.9D_{xy})^2$$

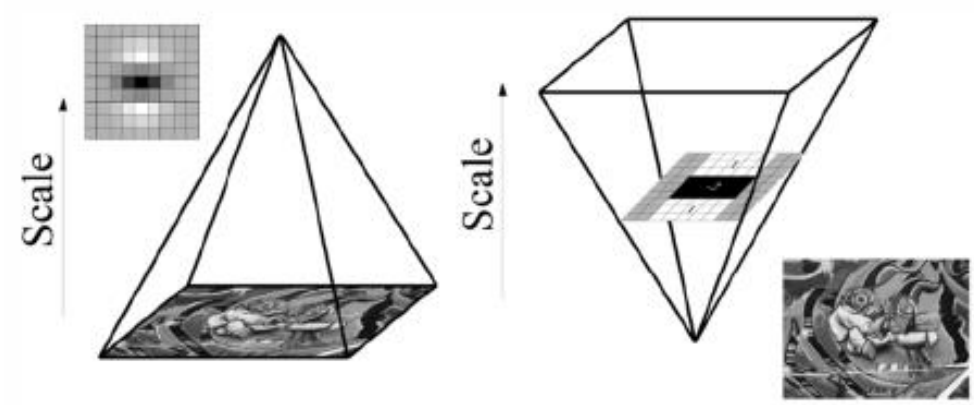


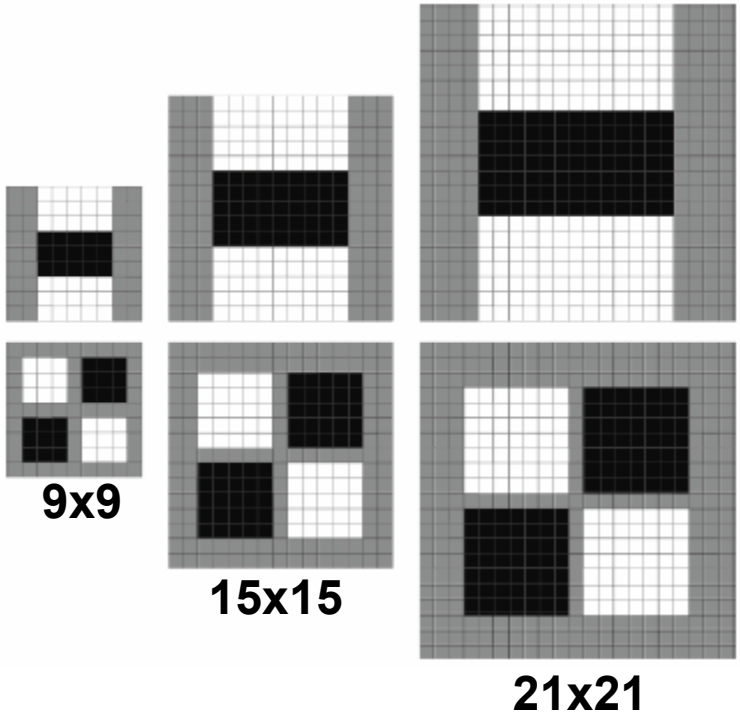
Gaussian

Laplacian

Changing scale

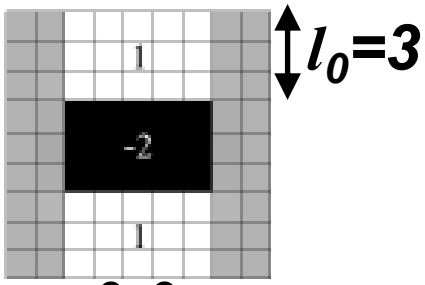
- **Integral image allows us to upsample the filter rather than downsample the image**



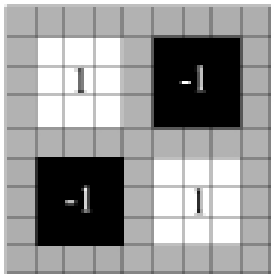


Changing scale
(within an octave)

- **For 9x9 filter, l_0 = 3**
  (length of positive or negative lobe in direction of derivative)

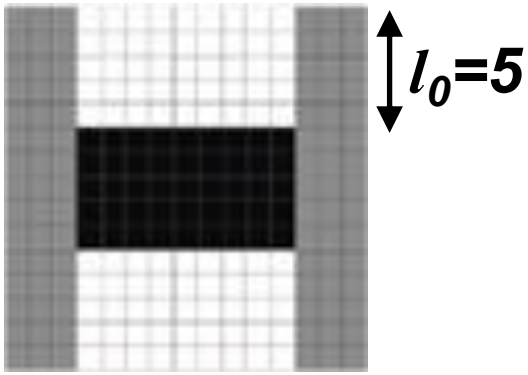


9x9



- **To keep central pixel, must increase l_0 by minimum of 2 pixels**
  → increase filter dimension by 6

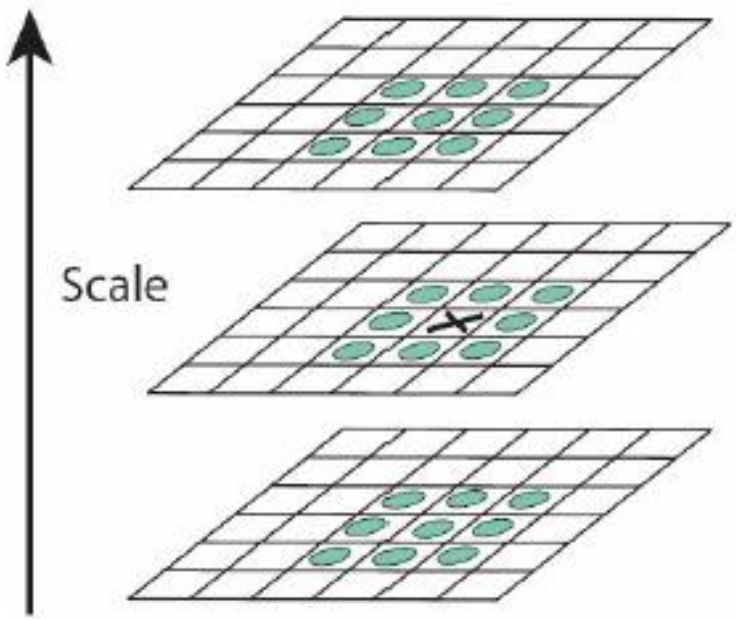
- **Therefore, sizes of filter: 9x9, 15x15, 21x21, 27x27**



15x15

Non-maximal suppression

- **Retain pixel only if greater than 26 neighbors in *x, y, s***



Interpolation

- **For each local maximum, need to interpolate to get true location (to overcome discretization effects)**
- **Hessian values:** $H(x, y, \sigma)$
- **Taylor expansion:**

$$H(\mathrm{x}) = H + \frac{\partial H}{\partial \mathrm{x}}^T \mathrm{x} + \frac{1}{2}\mathrm{x}^T \frac{\partial^2 H}{\partial \mathrm{x}^2}\mathrm{x}$$

- **Solution using Newton's method:**

$$\hat{x} = -\frac{\partial^2 H}{\partial \mathrm{x}^2}^{-1} \frac{\partial H}{\partial \mathrm{x}}$$

$$\frac{\partial^2 H}{\partial \mathrm{x}^2} = \begin{bmatrix} d_{xx} & d_{yx} & d_{sx} \\ d_{xy} & d_{yy} & d_{sy} \\ d_{xs} & d_{ys} & d_{ss} \end{bmatrix}$$

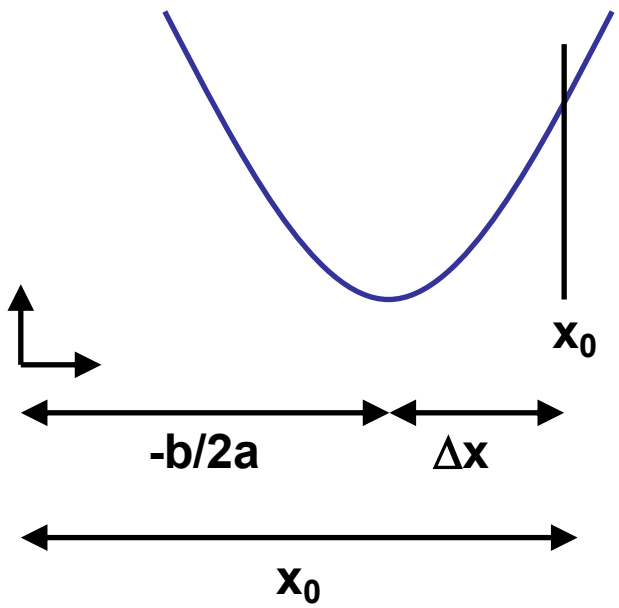
$$\frac{\partial H}{\partial \mathrm{x}} = \begin{bmatrix} d_x \\ d_y \\ d_s \end{bmatrix}$$

Interpolation in 1D

- **Newton's method:**
  - Given value at x_0, and derivatives at x_0, estimate parabola
  - Parabola is $y = f(x) = ax^2+bx+c$
  - Extremum of parabola (minimum if $a>0$, maximum if $a<0$) is given by $x^* = -b / 2a$
  - Repeat

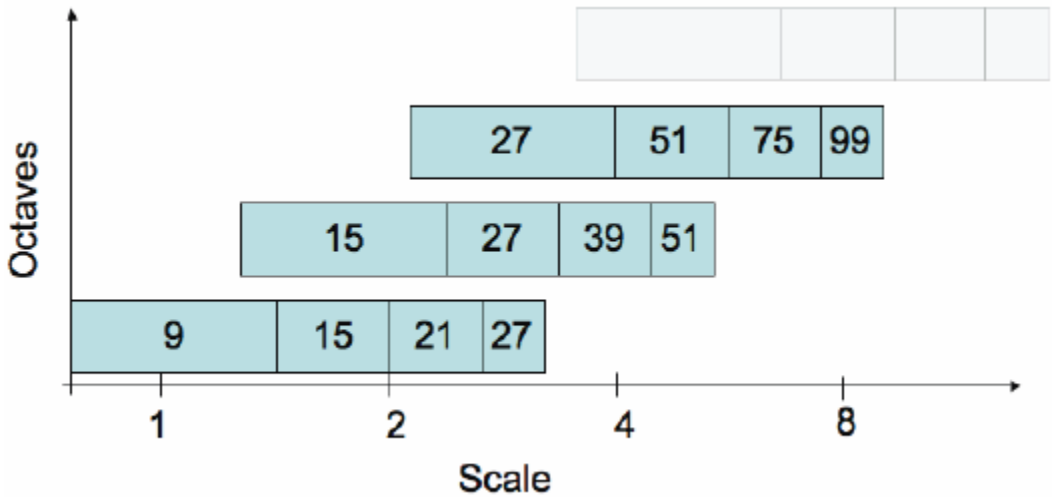
$f'(x) = 2ax+b$
$f''(x) = 2a$

Newton step:
$\Delta x = -f'(x) / f''(x)$
$= -(2ax_0+b) / (2a)$
$= -x_0 - b/2a$



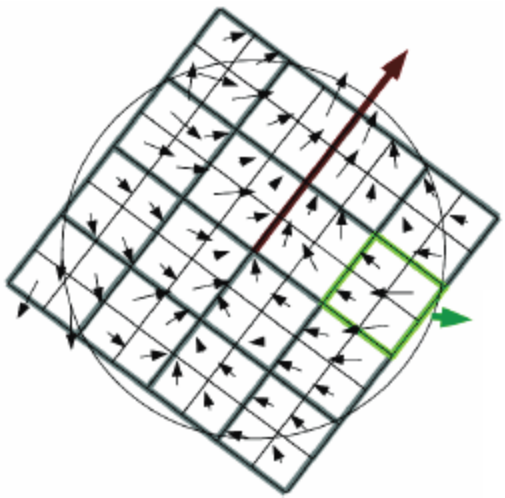
The next octaves

- First octave filter sizes: 9, 15, 21, 27
- Second octave sizes: 15, 27, 39, 51
  - Increase by 12 each time (not 6)
  - Spans from 21 (σ=1.2*21/9=2.8)
    to 45 (σ=1.2*45/9=6)
    (some overlap with first octave)
  - Ok to measure at every other pixel in image
    (saves computation, like downsampling)
- Third octave sizes:
  27, 51, 75, 99
  - Increase by 24
    each time
  - Spans from
    39 (σ=1.2*39/9=5.2)
    to 87 (σ=1.2*87/9=11.6)
  - Ok to measure at every
    4th pixel in image



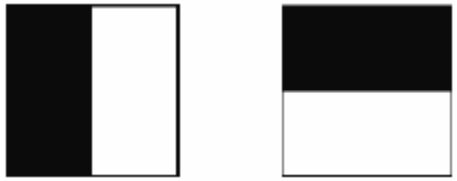
SURF descriptor

- **Once interest point has been found,**
  - **Place window around point**
  - **Divide into 4x4 subwindows**
  - **In each subwindow,**
    - **measure at 25 (5x5) places: dx and dy**
    - **sum over all 25 places to get 4 values:**

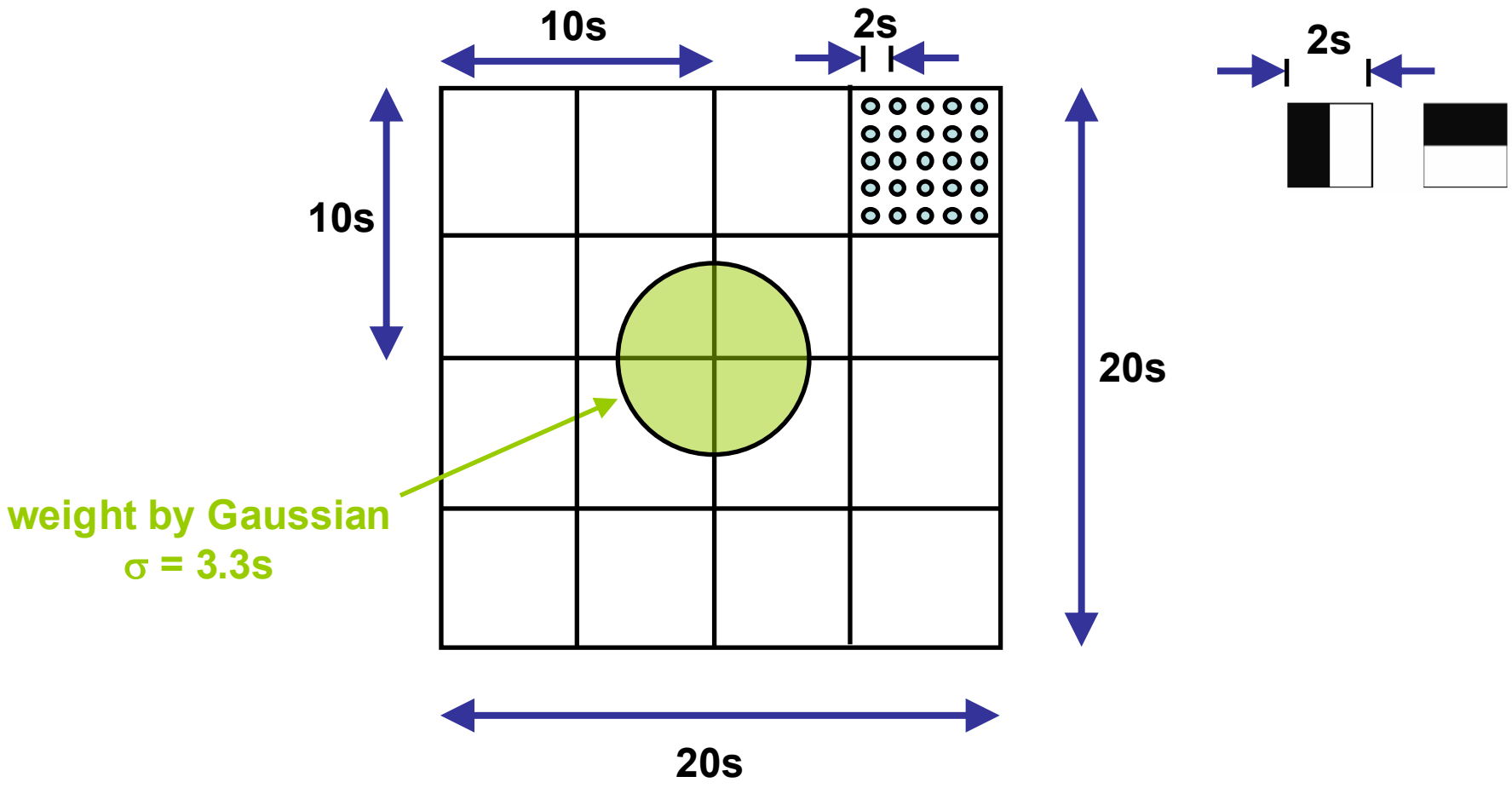


$$\sum dx$$
$$\sum |dx|$$
$$\sum dy$$
$$\sum |dy|$$

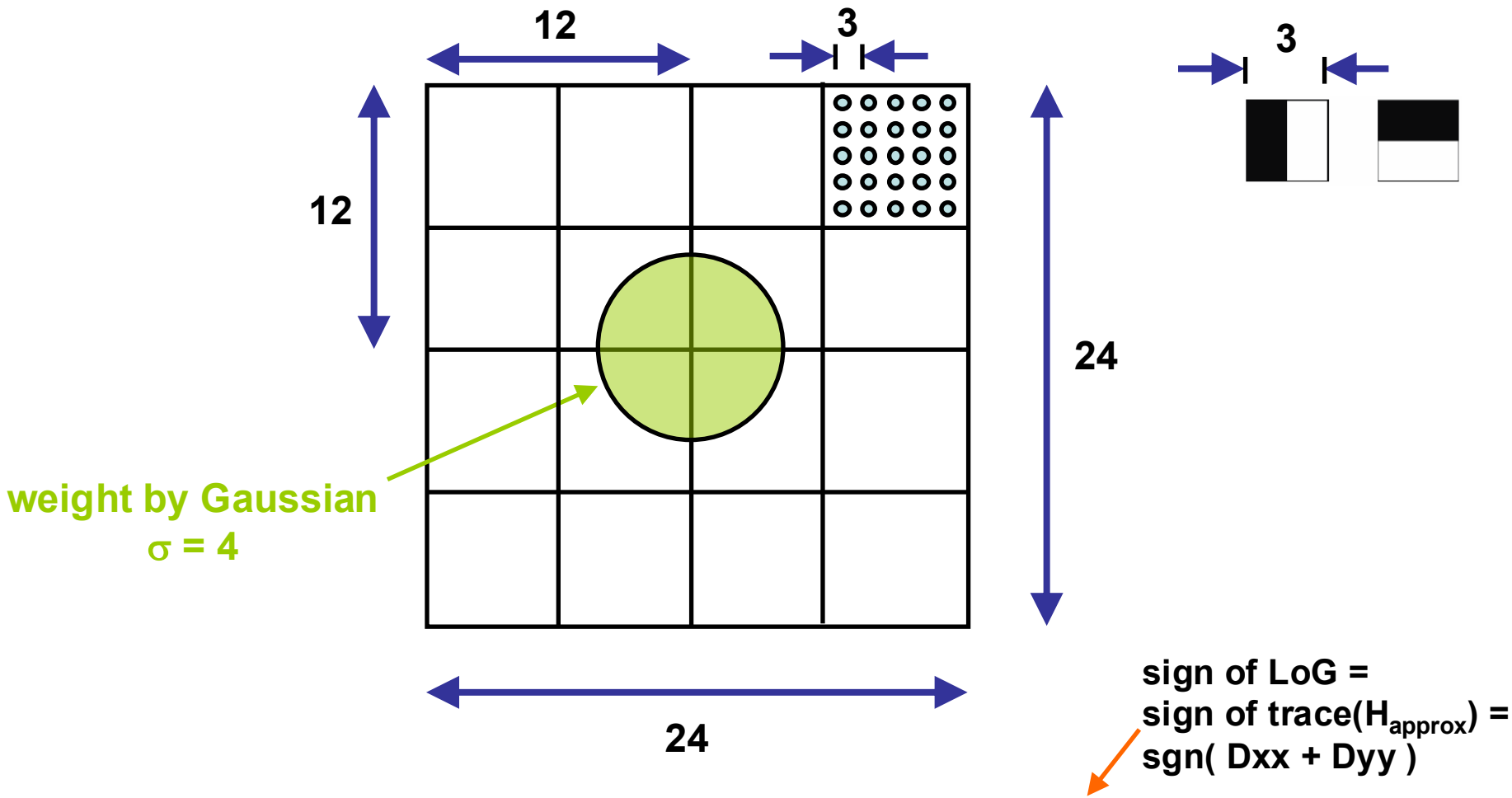
- **Note: Use Haar wavelets to measure differences (similar to gradients)**



Details



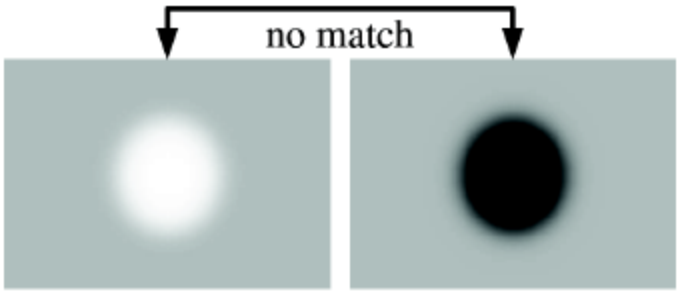
with s=1.2



→ 65 values per interest point (16 x 4 + 1)

sign of LoG

- **speeds up matching**



sgn(Dxx + Dyy)

Implementation results

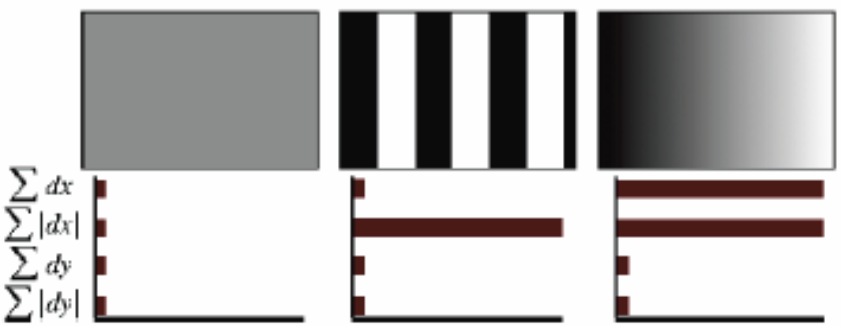
- Compare SIFT & SURF
  - SURF describes image faster than SIFT by 3 times
  - SURF is not as well as SIFT on invariance to illumination change and viewpoint change
  - http://cs.unc.edu/~ccwu/siftgpu
- Applications
- Challenge test
  - Object recognition (SLAM)
  - Face recognition
- Challenge test databases

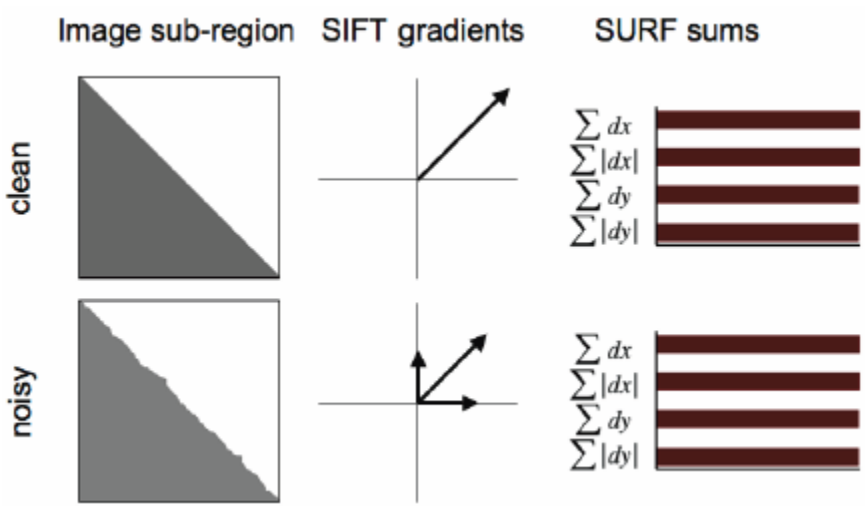
Analysis

- SURF is good at
  - handling serious blurring
  - handling image rotation
- SURF is poor at
  - handling viewpoint change
  - handling illumination change

- SURF is always better than the SIFT implemented by VGG but not the original SIFT

Why SURF is better than SIFT





Examples: Flowers

Examples: Tillman

SURF, 5 octaves, 4 scales per octave

Examples: Tillman

U-SURF, 1 octave, 4 scales per octave

Examples: Tillman

OpenCV's SURF